

7.1 Введение в структуры данных

7.1.1 Что такое структура данных?

Структура данных — это способ организовать и хранить данные так, чтобы с ними было удобно работать: получать элементы, искать их, добавлять, изменять и удалять. В Java к структурам данных относятся, например, **массивы** и **коллекции**.

7.1.2 Зачем вообще нужны структуры данных в программировании?

Они нужны потому, что программа почти всегда работает не с одним значением, а с **набором значений**. Например, с группой пользователей, списком заказов, массивом чисел, набором сообщений. Структура данных позволяет хранить такие данные не хаотично, а в понятной форме, с которой удобно работать в коде.

7.1.3 Чем структура данных отличается от просто набора переменных?

Просто набор переменных — это несколько отдельных значений, между которыми нет общей модели хранения. Структура данных объединяет элементы в одну сущность и задаёт правила работы с ними. То есть это уже не просто “несколько переменных”, а организованное хранилище данных.

7.1.4 Почему данные часто нужно хранить не по одному значению, а группами?

Потому что в реальных задачах почти всегда есть **много однотипных объектов**. Не один студент, а список студентов. Не один товар, а набор товаров. Не одна оценка, а вся история оценок. Хранить такие данные группами удобнее, чем заводить отдельную переменную под каждый элемент.

7.1.5 Какие базовые операции обычно есть у структуры данных?

Обычно у структуры данных есть операции **доступа, поиска, вставки, удаления** и **обхода элементов**. Конкретный набор и удобство этих операций зависят от самой структуры данных.

7.1.6 Почему разные структуры данных подходят под разные задачи?

Потому что они по-разному устроены внутри. Из-за этого одна структура лучше подходит для быстрого доступа по индексу, другая — для удобного добавления и удаления, третья — для хранения данных в определённом порядке. Универсальной структуры данных “на все случаи жизни” нет.

7.1.7 Что такое доступ, поиск, вставка и удаление?

Доступ — это получение нужного элемента из структуры данных.

Поиск — это нахождение элемента по какому-то признаку.

Вставка — это добавление нового элемента.

Удаление — это удаление уже существующего элемента.

Это базовые действия, ради которых структуры данных и используются.

7.1.8 Почему время выполнения операций зависит от выбранной структуры данных?

Потому что стоимость операций зависит от того, **как именно данные хранятся внутри**. В одной структуре доступ к элементу может быть очень удобным, а вставка неудобной. В другой — наоборот. Поэтому при выборе структуры данных всегда важно думать не только о том, **что хранить**, но и **что чаще делать с этими данными**.

7.2 Массивы как первая структура данных

7.2.1 Что такое массив в Java?

Массив — это объект, который хранит **фиксированное количество элементов одного типа**. После создания длина массива уже не меняется. В массиве можно хранить либо примитивные значения, либо ссылки на объекты.

7.2.2 Зачем нужен массив и когда его удобно использовать?

Массив удобен, когда нужно хранить **набор однотипных данных** и заранее понятно, сколько элементов должно быть. Это простая и базовая структура данных, с которой обычно начинают изучение хранения групп значений в Java.

7.2.3 Как создать массив в Java?

Массив можно создать через оператор `new`, указав тип элементов и размер. Можно также создать массив сразу с начальными значениями.

```
int[] numbers = new int[5];
```

```
String[] names = new String[3];
```

```
int[] scores = {10, 20, 30, 40};
```

При первом варианте создаётся массив заданной длины с начальными значениями по умолчанию. При втором — массив создаётся сразу с указанными элементами.

7.2.4 Почему все элементы массива имеют один и тот же тип?

Потому что массив в Java создаётся как структура с **одним типом элементов**. Когда мы объявляем массив, язык заранее знает, какие именно значения в нём допустимы. Это делает работу с массивом более строгой и предсказуемой.

7.2.5 Что такое индекс массива?

Индекс массива — это номер позиции элемента внутри массива. Именно по индексу программа понимает, к какому элементу нужно обратиться. В Java индексы массивов работают как целочисленные значения.

7.2.6 Почему индексация начинается с нуля?

В Java первый элемент массива имеет индекс 0, второй — 1, третий — 2 и так далее. Это правило языка. Для программиста здесь важно запомнить простую вещь: **первый элемент массива — всегда нулевой**.

7.2.7 Как получить элемент массива и изменить его значение?

Чтобы получить элемент массива, нужно указать имя массива и индекс в квадратных скобках. Чтобы изменить элемент, нужно обратиться к той же позиции и присвоить ей новое значение.

```
int[] numbers = {10, 20, 30};
```

```
int first = numbers[0];  
System.out.println(first); // 10
```

```
numbers[1] = 99;  
System.out.println(numbers[1]); // 99
```

Здесь `numbers[0]` — чтение элемента, а `numbers[1] = 99;` — изменение значения элемента по индексу.

7.2.8 Как узнать длину массива?

У массива есть специальное поле `length`, которое хранит его длину. Это именно поле, а не метод, поэтому скобки не нужны.

```
int[] numbers = {10, 20, 30, 40};
```

```
System.out.println(numbers.length); // 4
```

Через `length` можно узнать, сколько элементов помещается в массиве.

7.2.9 Почему размер массива нельзя изменить после создания?

Потому что массив в Java имеет **фиксированную длину**, которая задаётся в момент создания. Если нужен массив другого размера, создают **новый массив**.

```
int[] oldArray = {1, 2, 3};  
int[] newArray = new int[5];
```

То есть массив не “растягивается” автоматически. Для изменяемого размера обычно используют уже не массив, а коллекции, например `ArrayList`.

7.2.10 Какие значения по умолчанию получают элементы массива?

Когда создаётся новый массив, его элементы автоматически получают значения по умолчанию.

Для числовых типов это 0, для boolean — false, для char — нулевой символ, для ссылочных типов — null.

```
int[] numbers = new int[3];
boolean[] flags = new boolean[2];
String[] names = new String[2];
```

```
System.out.println(numbers[0]); // 0
System.out.println(flags[0]); // false
System.out.println(names[0]); // null
```

Это важная особенность массивов: их элементы инициализируются автоматически при создании массива.

7.2.11 Чем массив примитивов отличается от массива объектов?

В массиве примитивов хранятся **сами значения**, например int, double, char.

В массиве объектов хранятся **ссылки на объекты**, а не сами объекты.

```
int[] numbers = new int[2];
String[] words = new String[2];
```

```
System.out.println(numbers[0]); // 0
System.out.println(words[0]); // null
```

Поэтому в массиве объектов по умолчанию мы видим null, а в массиве примитивов — стандартное значение соответствующего примитивного типа.

7.2.12 Что происходит при выходе за границы массива?

Если обратиться к индексу меньше 0 или к индексу, который больше либо равен длине массива, Java выбросит исключение `ArrayIndexOutOfBoundsException`.

```
int[] numbers = {10, 20, 30};

System.out.println(numbers[3]); // ошибка
```

Это встроенная защита языка: Java не позволяет обращаться к несуществующим элементам массива.

7.3 От массивов к коллекциям

7.3.1 Какие проблемы у массивов возникают в реальных программах?

Главные проблемы массивов в реальных программах такие: у них **фиксированный размер**, они не умеют сами удобно расти и уменьшаться, а при частых изменениях содержимого с ними приходится делать много ручной работы. Массив хорош как простая базовая структура, но когда данные постоянно добавляются, удаляются и меняются по ходу работы программы, его возможностей быстро начинает не хватать.

7.3.2 Почему массив неудобен для частого добавления и удаления элементов?

Потому что у массива длина фиксирована. Если в нём “нет места”, нельзя просто дописать новый элемент — обычно нужно создать новый массив большего размера и перенести данные. При удалении тоже возникает неудобство: элементы часто приходится вручную сдвигать, чтобы не оставлять “дырку” посередине.

Пример неудобства с удалением из массива:

```
int[] numbers = {10, 20, 30, 40};
```

```
// Удалим элемент с индексом 1 (значение 20)
```

```
for (int i = 1; i < numbers.length - 1; i++) {  
    numbers[i] = numbers[i + 1];  
}
```

```
// Теперь массив стал таким: [10, 30, 40, 40]
```

```
// Лишнее значение в конце остается, и это уже неудобно
```

7.3.3 Почему программистам понадобились коллекции?

Коллекции понадобились потому, что программам очень часто нужно хранить **группы объектов** и выполнять над ними типовые операции: добавлять, удалять, искать, обходить, сортировать. Хотелось иметь не самодельные решения под каждую задачу, а готовую общую архитектуру с понятными интерфейсами и реализациями. Именно это и даёт Collections Framework.

7.3.4 Что значит, что коллекции являются абстракцией над хранением данных?

Это значит, что программист чаще работает не с внутренним устройством хранения, а с **общим интерфейсом поведения**. Например, он думает не “как именно лежат элементы в памяти”, а “мне нужен список”, “мне нужен набор без дубликатов”, “мне нужна очередь”. Внутреннюю реализацию уже берёт на себя конкретный класс коллекции.

7.3.5 Почему коллекции не магия, а обычные классы, написанные людьми?

Потому что коллекции в Java — это обычные интерфейсы и классы из `java.util`, у которых есть свои поля, методы и внутренняя логика. Они не “особая встроенная магия языка”, а обычный код библиотеки, просто очень хорошо продуманный и стандартизированный.

7.3.6 Когда массив все еще лучше коллекции?

Массив всё ещё удобен, когда размер данных **известен заранее**, когда нужна максимально простая структура, когда важна работа именно с **примитивами**, а не с объектами-обёртками, и когда не нужны гибкие операции добавления и удаления. Ещё массивы хороши как базовая низкоуровневая структура, поверх которой уже строятся более удобные коллекции.

7.4 Общее понимание коллекций в Java

7.4.1 Что такое Collections Framework в Java?

Collections Framework — это единая архитектура для представления и обработки групп объектов в Java. В неё входят **интерфейсы, реализации и алгоритмы** для работы с коллекциями. За счёт этого с разными коллекциями можно работать по общим правилам, не привязываясь слишком сильно к их внутреннему устройству.

7.4.2 Какие основные типы коллекций есть в Java?

На базовом уровне чаще всего выделяют **List, Set, Queue** и **Deque**. Отдельно очень важно помнить про **Map**: он тоже входит в Collections Framework, но формально не является подтипом Collection, потому что хранит не отдельные элементы, а пары **ключ-значение**.

7.4.3 Чем отличается Collection от Collections?

Collection — это интерфейс, то есть общий контракт для коллекций.

Collections — это служебный класс с **static-методами** для работы с коллекциями, например для сортировки, перемешивания, поиска и создания некоторых специальных обёрток. Эти вещи часто путают просто из-за похожих названий.

Пример:

```
List<String> names = new ArrayList<>();  
names.add("Аня");  
names.add("Борис");  
names.add("Вика");  
Collections.sort(names);  
System.out.println(names);
```


7.4.4 Какие базовые операции есть у большинства коллекций?

У большинства коллекций есть базовые операции вроде **добавления, удаления, проверки наличия элемента, получения размера и обхода элементов**. У конкретных типов коллекций могут добавляться свои специальные операции, но этот базовый набор встречается очень часто.

Пример:

```
Collection<String> words = new ArrayList<>();  
words.add("java");  
words.add("spring");  
words.remove("java");  
System.out.println(words.contains("spring")); // true  
System.out.println(words.size());           // 1
```

7.4.5 Как понять, какую коллекцию выбрать под задачу?

Нужно смотреть на то, **какие свойства тебе важны**. Если нужен порядок и доступ по индексу — обычно берут List. Если нужны уникальные элементы — Set. Если важна модель очереди — Queue или Deque. Если нужно хранить значения по ключу — Map. После этого уже выбирают конкретную реализацию с подходящими характеристиками.

7.4.6 Почему одна и та же операция в разных коллекциях работает по-разному?

Потому что одинаковые по смыслу операции могут опираться на **разное внутреннее устройство**. Например, `get(i)` в списках есть у List, но стоимость такого доступа зависит от реализации: для одних списков он очень удобен, для других заметно хуже. То же самое касается вставки, удаления и обхода.

7.5 Дженерики и типобезопасность коллекций

7.5.1 Зачем коллекциям нужны дженерики?

Дженерики нужны, чтобы коллекция хранила **элементы конкретного типа**, а ошибки по типам находились ещё **на этапе компиляции**, а не только во время выполнения. Это делает код безопаснее, понятнее и удобнее.

7.5.2 Что дает запись вроде List<String>?

Запись List<String> означает: это список, в котором ожидаются элементы типа String. Благодаря этому компилятор проверяет, что в список не пытаются положить что-то неподходящее, а при чтении элементов не нужно делать лишние приведения типов.

Пример:

```
List<String> names = new ArrayList<>();
```

```
names.add("Анна");
```

```
// names.add(123); // ошибка компиляции
```

```
String name = names.get(0);
```

```
System.out.println(name);
```

7.5.3 Что такое типобезопасность коллекций?

Типобезопасность означает, что коллекция используется так, чтобы в ней оказывались только допустимые элементы, а нарушения этого правила обнаруживались как можно раньше. В случае дженериков Java проверяет это уже на этапе компиляции, и код становится надёжнее.

7.5.4 Почему без дженериков работать с коллекциями опаснее и менее удобно?

Потому что без дженериков коллекция может принимать объекты разных типов, и ошибки часто всплывают уже позже, когда программист читает элемент и пытается привести его к ожидаемому типу. Кроме того, код становится менее удобным: приходится чаще делать явные приведения, а это и лишний шум, и риск ошибок.

Пример проблемы без дженериков:

```
List list = new ArrayList();
```

```
list.add("hello");
```

```
list.add(123);
```

```
String text = (String) list.get(1); // ClassCastException во время выполнения
```

7.5.5 Что такое raw type и почему его лучше избегать?

Raw type — это использование generic-класса **без указания параметра типа**, например просто `List` вместо `List<String>`. Так делать можно ради совместимости со старым кодом, но это убирает большую часть пользы дженериков: компилятор хуже проверяет типы, появляются предупреждения и повышается риск ошибок во время выполнения.

Пример:

```
List raw = new ArrayList();
```

```
raw.add("text");
```

```
raw.add(100);
```

7.6 Интерфейс List

7.6.1 Что такое List и для каких задач он нужен?

List — это коллекция, которая хранит элементы в **определённом порядке** и поддерживает **доступ по индексу**. Он нужен, когда важна последовательность элементов, когда могут быть дубликаты и когда программисту удобно работать с позициями элементов.

7.6.2 Чем List отличается от массива?

И массив, и List хранят элементы по порядку и позволяют обращаться к ним по индексу. Но массив имеет **фиксированный размер** и является частью языка, а List — это интерфейс Collections Framework, у которого есть разные реализации, и многие из них умеют удобно работать с изменением количества элементов. Кроме того, List хранит объекты, а не примитивы напрямую.

Пример:

```
int[] array = {10, 20, 30};
```

```
List<Integer> list = new ArrayList<>();
```

```
list.add(10);
```

```
list.add(20);
```

```
list.add(30);
```

```
list.add(40); // список можно удобно расширять
```

7.6.3 Почему List хранит элементы в определенном порядке?

Потому что сам смысл List — это **упорядоченная последовательность** элементов. Для такого типа коллекции важна позиция элемента: первый, второй, третий и так далее. Именно поэтому у List есть операции, связанные с индексами и положением элементов.

7.6.4 Почему в List можно хранить дубликаты?

Потому что List моделирует именно **последовательность**, а не набор уникальных значений. Если в последовательности одно и то же значение встречается несколько раз, это считается нормальным. Запрет дубликатов — это уже задача Set, а не List.

Пример:

```
List<String> names = new ArrayList<>();  
names.add("Анна");  
names.add("Анна");  
names.add("Борис");
```

```
System.out.println(names); // [Анна, Анна, Борис]
```

7.6.5 Какие основные операции есть у List?

У List есть операции **добавления, удаления, получения по индексу, замены элемента, поиска, получения размера** и **обхода**. Отдельно важны именно позиционные операции, потому что они отличают список от более общего Collection.

Пример:

```
List<String> fruits = new ArrayList<>();  
fruits.add("яблоко");  
fruits.add("банан");  
fruits.add("груша");  
System.out.println(fruits.get(1)); // банан  
fruits.set(1, "апельсин");  
fruits.remove(0);  
System.out.println(fruits);    // [апельсин, груша]  
System.out.println(fruits.size()); // 2
```

7.6.6 Когда List особенно удобен?

List особенно удобен, когда нужен **сохранённый порядок элементов**, когда важен **доступ по позиции**, когда возможны **повторы** и когда данные логически представляют именно последовательность: список задач, сообщения чата, шаги инструкции, товары в корзине, оценки студента.

7.7 ArrayList

7.7.1 Что такое ArrayList и на какой идее он построен?

ArrayList — это список, построенный на идее **динамического массива**. Снаружи он выглядит как обычный List, но внутри опирается на массив, который при необходимости расширяется. Поэтому он хорошо подходит для хранения элементов в порядке и для быстрого доступа по индексу.

7.7.2 Как ArrayList хранит элементы внутри?

Внутри у него есть массив `elementData` и отдельное поле `size`. Массив хранит ячейки, а `size` показывает, сколько из них реально занято. То есть список не равен массиву целиком: часть массива может быть пустой, но уже выделенной “про запас”. В исходниках это видно по полям `elementData` и `size`.

Упрощённо идея такая:

```
class ArrayList<E> {  
    Object[] elementData;  
    int size;  
}
```

7.7.3 Что происходит внутри при добавлении элемента в ArrayList?

При обычном `add(e)` список сначала проверяет, хватает ли места во внутреннем массиве. Если места нет, он расширяет массив. После этого кладёт элемент в позицию `size` и увеличивает `size` на 1. В реальной реализации это видно по вызову `add(e, elementData, size)` и записи элемента в конец занятой части массива.

Упрощённо по реальной логике:

```
public boolean add(E e) {  
    if (size == elementData.length) {  
        grow();  
    }  
    elementData[size] = e;  
    size++;  
    return true;  
}
```

7.7.4 Что происходит, когда внутренний массив ArrayList заполняется?

Когда массив заполнен, ArrayList создаёт **новый массив большей длины** и копирует туда старые элементы. В текущей реализации рост идёт не “на 1 элемент”, а с запасом: берётся старая ёмкость и добавляется ещё часть сверху. Если список был пустым в дефолтном состоянии, при первом расширении он получает стартовую ёмкость 10. Это видно по `grow(int minCapacity)`, `Arrays.copyOf(...)` и `DEFAULT_CAPACITY = 10`.

Упрощённо:

```
private void grow() {  
    int oldCapacity = elementData.length;  
    int newCapacity = oldCapacity + (oldCapacity / 2);  
  
    if (oldCapacity == 0) {  
        newCapacity = 10;  
    }  
  
    elementData = Arrays.copyOf(elementData, newCapacity);  
}
```

7.7.5 Чем size отличается от capacity?

`size` — это сколько элементов реально лежит в списке сейчас.

`capacity` — это сколько элементов помещается во внутренний массив **без нового расширения**.

Например, если массив внутри длиной 10, а в списке реально 3 элемента, то `size` = 3, а `capacity` = 10.

7.7.6 Как работает get в ArrayList?

`get(index)` проверяет, что индекс допустим, и сразу возвращает элемент из массива по этому индексу. Именно поэтому доступ по индексу у ArrayList сильный: внутри не нужно идти по цепочке узлов, как в связном списке. В исходниках это очень прямолинейно: после проверки идёт возврат элемента из массива.

Упрощённо:

```
public E get(int index) {  
    checkIndex(index);  
    return (E) elementData[index];  
}
```

```
}
```

7.7.7 Как работает add по индексу в ArrayList?

Если вставка идёт не в конец, а в середину, ArrayList сначала убеждается, что есть место, а потом **сдвигает хвост массива вправо**. После этого новый элемент записывается в нужную позицию. В реальной реализации для сдвига используется `System.arraycopy(...)`.

Упрощённо:

```
public void add(int index, E element) {  
    if (size == elementData.length) {  
        grow();  
    }  
}
```

```
System.arraycopy(elementData, index,  
                 elementData, index + 1,  
                 size - index);
```

```
elementData[index] = element;  
size++;
```

```
}
```

7.7.8 Как работает remove в ArrayList?

При удалении по индексу список сначала запоминает удаляемое значение, потом **сдвигает элементы справа влево**, уменьшает `size` и зануляет последнюю теперь уже ненужную ячейку. В реальной реализации это делается через `fastRemove(...)` и `System.arraycopy(...)`.

Упрощённо:

```
public E remove(int index) {
```

```
    E oldValue = (E) elementData[index];
```

```
System.arraycopy(elementData, index + 1,  
                 elementData, index,  
                 size - index - 1);
```

```
size--;
```



```
elementData[size] = null;
```

```
return oldValue;
```

```
}
```

7.7.9 Почему вставка и удаление в середину ArrayList могут быть дорогими?

Потому что в этих операциях нужно двигать целый хвост массива. Чем ближе операция к началу и чем больше список, тем больше элементов придётся сдвинуть. Поэтому ArrayList очень хорош в чтении по индексу и добавлении в конец, но заметно хуже себя чувствует при частых изменениях в середине.

7.7.10 Когда ArrayList особенно хорош, а когда неудобен?

ArrayList особенно хорош, когда нужен быстрый доступ по индексу, частый обход элементов и частое добавление в конец. Неудобен он тогда, когда в структуре постоянно идут вставки и удаления в начале или середине. Для таких сценариев массивная природа списка начинает мешать.

7.8 LinkedList

7.8.1 Что такое LinkedList?

LinkedList — это список, построенный не на массиве, а на **цепочке узлов**. В Java это **двусвязный список**: каждый узел знает и следующий элемент, и предыдущий. Поэтому у него совсем другая модель хранения, чем у ArrayList.

7.8.2 Чем связный список отличается от массива по идее хранения?

Массив хранит элементы в одном сплошном блоке по индексам. Связный список хранит элементы как отдельные узлы, соединённые ссылками. Из-за этого у массива сильная сторона — быстрый доступ по индексу, а у связного списка — удобная работа через перевязку ссылок между соседними узлами.

7.8.3 Что хранит узел LinkedList?

Узел хранит сам элемент и две ссылки: на следующий и предыдущий узел. В реальной реализации это видно в классе `Node<E>` с полями `item`, `next` и `prev`.

Упрощённо:

```
private static class Node<E> {  
    E item;  
    Node<E> next;  
    Node<E> prev;  
}
```

7.8.4 Как LinkedList хранит первый и последний элемент?

У списка есть отдельные ссылки `first` и `last`. Это значит, что он сразу знает голову и хвост списка и не обязан каждый раз обходить всю цепочку, чтобы добраться до краёв. В исходниках это отдельные поля рядом с `size`.

7.8.5 Как работает добавление в начало и в конец LinkedList?

При добавлении в начало создаётся новый узел, его `next` указывает на старый первый узел, а `first` переносится на новый узел. Если список был пуст, то и `last` тоже начинает указывать на этот же узел.

При добавлении в конец всё зеркально: новый узел связывается со старым хвостом, а `last` переносится на него. В реальной реализации это методы `linkFirst(e)` и `linkLast(e)`.

Упрощённо:

```
private void linkFirst(E e) {
    Node<E> f = first;

    Node<E> newNode = new Node<>(null, e, f);

    first = newNode;

    if (f == null) {
        last = newNode;
    } else {
        f.prev = newNode;
    }

    size++;
}

private void linkLast(E e) {
    Node<E> l = last;

    Node<E> newNode = new Node<>(l, e, null);

    last = newNode;

    if (l == null) {
        first = newNode;
    } else {
        l.next = newNode;
    }

    size++;
}
```

7.8.6 Как работает доступ по индексу в LinkedList?

get(index) не может взять элемент напрямую, потому что внутри нет массива. Он вызывает внутренний поиск узла по индексу, а потом возвращает item из найденного узла. В реализации это выглядит как вызов node(index).item.

Упрощённо:

```
public E get(int index) {
    return node(index).item;
}
```

7.8.7 Почему LinkedList плохо подходит для частого случайного доступа?

Потому что для `get(5000)` ему обычно нужно пройти по ссылкам до нужного узла. Даже при оптимизации “идти от ближайшего края” это всё равно не прямой доступ, а обход. В исходниках видно, что `node(index)` выбирает: идти от `first` или от `last`, смотря что ближе.

Упрощённо логика поиска узла такая:

```
Node<E> node(int index) {  
    if (index < size / 2) {  
        Node<E> x = first;  
        for (int i = 0; i < index; i++) {  
            x = x.next;  
        }  
        return x;  
    } else {  
        Node<E> x = last;  
        for (int i = size - 1; i > index; i--) {  
            x = x.prev;  
        }  
        return x;  
    }  
}
```

7.8.8 В каких задачах LinkedList может быть полезен?

LinkedList может быть полезен там, где структура данных по смыслу ближе к очереди, деку или последовательности с активной работой у краёв. У него естественно устроены операции на начале и конце списка. Но как обычная “универсальная замена ArrayList” он часто проигрывает.

7.9 Сравнение ArrayList и LinkedList

7.9.1 Чем ArrayList и LinkedList отличаются по внутреннему устройству?

ArrayList хранит элементы во внутреннем массиве `Object[]` и отдельно считает `size`.

LinkedList хранит элементы как цепочку узлов `Node`, где у каждого узла есть ссылки на соседей, а сам список хранит `first` и `last`. То есть снаружи оба — списки, а внутри устроены совсем по-разному.

7.9.2 В каких операциях ArrayList обычно выигрывает у LinkedList?

Обычно он выигрывает в доступе по индексу, в плотном обходе элементов и в типичном сценарии “добавляю в конец и читаю по позициям”. Для этого массивная структура подходит лучше. У самого ArrayList прямо отмечено, что `get` работает за постоянное время, а у LinkedList индексные операции могут идти пропорционально индексу.

7.9.3 В каких операциях LinkedList может быть удобнее?

Он может быть удобнее для операций у начала и конца списка, а также в сценариях очереди и дека. Но важно не переоценивать это: если тебе всё равно сначала нужно долго искать место по индексу, выгода может исчезнуть. Сам по себе связный список не делает любую вставку “автоматически быстрой” — многое зависит от того, как ты вообще дошёл до нужной позиции.

7.9.4 Почему асимптотика не всегда полностью отражает реальную производительность?

Потому что асимптотика показывает общую идею роста стоимости, но не показывает цену прохода по ссылкам, копирования непрерывных блоков памяти, работу кэша процессора и постоянные накладные расходы. Поэтому в реальной жизни ArrayList очень часто оказывается быстрее, чем многие ожидают по одной только теории про связные списки. Даже в описании ArrayList отдельно отмечено, что у него маленький постоянный коэффициент по сравнению с LinkedList.

7.10 equals, hashCode, Comparable и Comparator в коллекциях

7.10.1 Что делает equals() при работе с коллекциями?

equals() определяет, считаются ли две коллекции равными. Но смысл равенства зависит от типа коллекции.

Для List равенство означает: одинаковый размер и одинаковые элементы **в том же порядке**.

Для Set равенство означает: одинаковый набор элементов, порядок не важен.

Для Map равенство означает: одинаковые пары ключ–значение.

7.10.2 Что делает hashCode() при работе с коллекциями?

hashCode() даёт числовое представление коллекции, согласованное с её equals().

У List хеш зависит от элементов и их порядка.

У Set хеш считается по элементам без идеи порядка.

У Map — по парам entrySet().

Главная мысль одна: если две коллекции равны по equals(), их hashCode() тоже должен совпадать.

7.10.3 Почему для коллекций важно не нарушать контракт

equals()/hashCode()?

Потому что хеш-структуры, например HashSet и HashMap, используют оба метода вместе. В реальной реализации HashSet внутри опирается на HashMap, а HashMap сначала считает хеш ключа, потом ищет нужную корзину, а потом уже сравнивает ключи на равенство. Если равные объекты дают разные хеши, коллекция начинает вести себя неправильно: дубликаты могут “просачиваться”, поиск и удаление начинают ломаться.

Упрощённо суть там такая:

```
// HashSet

public boolean add(E e) {
    return map.put(e, PRESENT) == null;
}

// HashMap

public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

```
// внутри HashMap при поиске существующего ключа
if (storedHash == hash &&
    (storedKey == key || key.equals(storedKey))) {
    // это тот же ключ
}
```

7.10.4 Что такое Comparable?

Comparable — это интерфейс для **естественного порядка** объектов. Если класс его реализует, он сам умеет сравнивать свои объекты через `compareTo(...)`. Благодаря этому такие объекты можно сортировать без отдельного компаратора, а также использовать в отсортированных структурах, где нужен естественный порядок.

Пример:

```
class User implements Comparable<User> {
    private String name;

    @Override
    public int compareTo(User other) {
        return this.name.compareTo(other.name);
    }
}
```

7.10.5 Что такое Comparator?

Comparator — это отдельный объект, который задаёт **внешнее правило сравнения**. Он нужен, когда сортировку не хочется вшивать в сам класс или когда для одного и того же типа нужны разные способы сравнения.

Пример:

```
Comparator<User> byName = (a, b) -> a.getName().compareTo(b.getName());
Comparator<User> byAge = (a, b) -> Integer.compare(a.getAge(), b.getAge());
```

7.10.6 Чем Comparable отличается от Comparator?

Comparable — это сравнение **изнутри самого класса**: класс говорит, какой у него естественный порядок.

Comparator — это сравнение **снаружи**, отдельным объектом.

Если у типа один естественный порядок, часто подходит Comparable. Если способов сортировки несколько, обычно удобнее Comparator.

7.10.7 Почему compareTo и equals не должны противоречить друг другу?

Потому что отсортированные коллекции и карты опираются на результат сравнения. Для Comparable прямо рекомендуется, чтобы `x.compareTo(y) == 0` означало то же самое, что `x.equals(y)`. Для Comparator говорится то же самое через "consistent with equals". Если это нарушить, отсортированные Set и Map начинают вести себя странно: например, могут считать два неравных по `equals()` объекта одним и тем же элементом или, наоборот, пустить в множество то, что по смыслу должно считаться дублем.

7.11 Интерфейс Map

7.11.1 Что такое Map?

Map — это структура, которая хранит данные в виде связок **ключ** → **значение**. У каждого ключа есть соответствующее значение, и по ключу можно быстро получить то, что с ним связано. Это не просто “список элементов”, а отдельная модель хранения: один объект используется как идентификатор, другой — как данные, которые за ним стоят.

7.11.2 Чем Map отличается от Collection?

Collection — это коллекция отдельных элементов. Map хранит не отдельные элементы, а **пары ключ-значение**, поэтому Map вообще не является подтипом Collection. Вместо этого у него есть три представления содержимого: набор ключей, коллекция значений и набор пар entry.

7.11.3 Что такое ключ и значение?

Ключ — это то, по чему мы ищем данные. **Значение** — это то, что хранится под этим ключом. Например, в карте пользователей ключом может быть `userId`, а значением — объект `User`. Важно, что логика Map всегда строится именно вокруг доступа по ключу.

7.11.4 Почему ключи в Map должны быть уникальными?

Потому что сама идея Map такая: один ключ указывает максимум на одно текущее значение. Если бы одинаковый ключ мог встречаться несколько раз, было бы непонятно, что именно возвращать по этому ключу. Поэтому интерфейс Map прямо задаёт правило: карта не может содержать дубликаты ключей.

7.11.5 Какие основные операции есть у Map?

Базовые операции у Map — это `put`, `get`, `remove`, `containsKey`, `containsValue`, `size`, `clear`, а также получение представлений через `keySet()`, `values()` и `entrySet()`. Это покрывает основные сценарии: добавить пару, найти по ключу, удалить по ключу, проверить наличие и перебрать содержимое.

7.11.6 Где Map особенно полезна в реальном коде?

Map особенно полезна там, где нужен быстрый доступ по идентификатору: кэш по `id`, справочник кодов, индекс объектов по логину, хранение настроек по имени параметра, группировка данных по ключу, частотные словари, таблицы соответствий. Когда в задаче есть естественная модель “по этому ключу хочу сразу получить связанное значение”, Map обычно подходит лучше списка.

7.12 HashMap

7.12.1 Что такое HashMap?

HashMap — это реализация Map, построенная на **хеш-таблице**. Она поддерживает `null` как ключ и как значение, не гарантирует порядок элементов и обычно даёт очень быстрые `get` и `put`, если хеши ключей распределены нормально.

7.12.2 Что хранится внутри HashMap на базовом уровне?

На базовом уровне внутри HashMap есть массив бакетов `table`, размер `size`, порог расширения `threshold` и `loadFactor`. То есть это не “магическая карта”, а таблица, где каждая ячейка массива хранит либо пустоту, либо цепочку/дерево записей.

7.12.3 Что такое бакет в HashMap?

Бакет — это одна ячейка внутреннего массива таблицы. Именно в неё попадает ключ после вычисления хеша и индекса. Если несколько ключей попали в одну и ту же ячейку, внутри бакета уже хранятся несколько записей, и тогда нужно разруливать коллизию.

7.12.4 Что такое узел записи в HashMap?

Запись в HashMap — это объект узла `Node`, который хранит хеш, ключ, значение и ссылку на следующий узел. То есть один элемент карты — это не “голый ключ”, а полноценная структура: `hash`, `key`, `value`, `next`.

7.12.5 Как HashMap вычисляет место для хранения ключа?

Сначала она получает хеш ключа через внутреннюю функцию `hash(key)`. Эта функция не просто берёт `hashCode()`, а ещё смешивает биты, чтобы лучше распределять ключи. Потом индекс бакета вычисляется как $(n - 1) \& \text{hash}$, где n — длина таблицы. Благодаря тому, что длина таблицы — степень двойки, такое вычисление получается очень дешёвым.

Упрощённо по сути реализации:

```
int hash = spread(key.hashCode());
int index = (table.length - 1) & hash;
```

7.12.6 Какую роль играют `hashCode()` и `equals()` в HashMap?

`hashCode()` нужен, чтобы выбрать бакет. `equals()` нужен, чтобы уже внутри бакета точно понять, это тот же ключ или нет. В реальном коде HashMap сначала смотрит на хеш, а потом сравнивает ключи через `==` и `equals()`. То есть схема всегда такая: сначала грубое попадание по хешу, потом точная проверка равенства.

7.12.7 Как работает put в HashMap?

`put(key, value)` сначала вызывает внутренний `putVal(hash(key), key, value, ...)`. Дальше логика такая: если таблицы ещё нет — создаётся; если нужный бакет пуст — туда кладётся новый узел; если в бакете уже есть записи, карта либо находит существующий ключ и обновляет значение, либо добавляет новый узел в цепочку, либо работает с деревом, если бакет уже `treeified`. После вставки `size` увеличивается, и если он превысил `threshold`, запускается `resize()`.

Упрощённо по сути исходников:

```
V put(K key, V value) {
    return putVal(hash(key), key, value);
}

V putVal(int hash, K key, V value) {
    if (table is null) resize();

    int index = (table.length - 1) & hash;

    if (table[index] is empty) {
        table[index] = new Node(hash, key, value, null);
    } else {
        // ищем тот же ключ в цепочке или дереве
        // если нашли — обновляем value
        // если не нашли — добавляем новую запись
    }

    size++;
    if (size > threshold) resize();
}
```

7.12.8 Как работает get в HashMap?

`get(key)` вызывает внутренний `getNode(key)`. Там снова вычисляется хеш и индекс бакета. Потом `HashMap` проверяет первый узел бакета, а если не совпало — идёт дальше по цепочке или по дереву. Если нужный ключ найден, возвращается его `value`, если нет — `null`.

Упрощённо:

```

V get(Object key) {
    Node node = getNode(key);
    return node == null ? null : node.value;
}

Node getNode(Object key) {
    int hash = hash(key);
    int index = (table.length - 1) & hash;

    Node first = table[index];
    // проверяем first
    // если не он — идём по next или в дерево
}

```

7.12.9 Что происходит при коллизии в HashMap?

Коллизия — это когда разные ключи попадают в один и тот же бакет. Тогда HashMap не теряет старую запись, а хранит несколько записей в одном бакете. В обычном случае это цепочка узлов через `next`; в перегруженных бакетах — дерево. Поэтому коллизия сама по себе нормальна, вопрос только в том, насколько часто она происходит.

7.12.10 Что такое load factor и зачем он нужен?

`load factor` — это коэффициент заполнения таблицы. Он определяет, насколько полной карта может стать до расширения. По умолчанию это 0.75, и это компромисс между памятью и скоростью: слишком маленький коэффициент даёт лишний расход памяти, слишком большой увеличивает цену поиска.

7.12.11 Что такое resize и когда он происходит?

`resize` — это расширение внутренней таблицы HashMap. Оно запускается, когда число записей становится больше `threshold`, а `threshold` связан с ёмкостью таблицы и `loadFactor`. В обычном сценарии новая таблица получается примерно вдвое больше старой.

7.12.12 Почему HashMap перераспределяет элементы при расширении?

Потому что при новой длине таблицы меняются допустимые индексы бакетов. После расширения старые записи уже нельзя оставить “как есть” в старых ячейках: нужно заново разложить их по новой таблице. В реализации прямо видно, что элементы либо остаются на старом индексе, либо переезжают на индекс `j + oldCap`.

7.12.13 Почему в современных реализациях HashMap бакет может превращаться в дерево?

Потому что длинная цепочка в одном бакете начинает ухудшать поиск. Чтобы деградация не становилась слишком болезненной, при достаточно большом числе узлов бакет переводится в дерево. В текущей реализации порог `treeify` — 8 элементов, но дерево вообще начинают использовать только если сама таблица уже не слишком маленькая; иначе карта предпочитает сначала расшириться.

7.12.14 Почему плохой hashCode () ухудшает производительность Map?

Потому что плохой хеш загоняет слишком много ключей в одни и те же бакеты. Тогда вместо почти прямого доступа карта всё чаще вынуждена проходить по цепочке или дереву. В описании HashMap это проговаривается прямо: много ключей с одинаковым hashCode () — верный способ замедлить работу хеш-таблицы.

7.12.15 Почему mutable-ключи опасны для HashMap?

Потому что если объект уже лежит в карте как ключ, а потом у него меняются поля, участвующие в `equals ( )` или `hashCode ( )`, карта фактически “теряет” этот ключ. Он уже лежит в старом бакете, рассчитанном по старому состоянию, а искать его потом будут по новому хешу и новому равенству. Интерфейс Map отдельно предупреждает: изменяемые ключи требуют большой осторожности.

7.12.16 Можно ли использовать null как ключ и как значение в HashMap?

Да. HashMap разрешает `null` и как ключ, и как значение. Но из-за этого `get (key)` со значением `null` не позволяет сам по себе понять, ключа вообще нет или он есть и просто хранит `null`; для различения используют `containsKey (key)`.

7.12.17 Почему порядок элементов в HashMap не гарантирован?

Потому что HashMap устроен вокруг бакетов, хешей, коллизий и расширений, а не вокруг стабильного пользовательского порядка. В её описании прямо сказано, что она не делает гарантий по порядку, и порядок может меняться со временем. Поэтому на порядок элементов в HashMap рассчитывать нельзя.

7.12.18 Чем в общих чертах HashMap отличается от LinkedHashMap и TreeMap?

HashMap — это обычная хеш-таблица без гарантий по порядку. LinkedHashMap — это HashMap плюс двусвязный список записей, поэтому у неё есть предсказуемый `encounter order`, обычно порядок вставки. TreeMap — это уже не хеш-таблица, а красно-чёрное дерево, поэтому она хранит ключи в отсортированном порядке и даёт гарантированную логарифмическую стоимость `get`, `put`, `remove`.

7.13 Интерфейс Set

7.13.1 Что такое Set?

Set — это коллекция без дубликатов. Её идея — моделировать математическое множество: элемент либо есть, либо его нет, но второй “такой же” элемент добавлять нельзя.

7.13.2 Чем Set отличается от List?

List — это упорядоченная последовательность, где важна позиция элемента и разрешены повторы. Set — это структура, где важна уникальность элементов, а индексного доступа вообще нет. То есть List отвечает на вопрос “что лежит на позиции 5?”, а Set — на вопрос “есть ли у меня такой элемент вообще?”.

7.13.3 Почему Set не допускает дубликаты?

Потому что это часть самого контракта Set. Формально множество не должно содержать пару элементов e_1 и e_2 , для которых $e_1.equals(e_2)$. Для `null` тоже действует ограничение: максимум один `null`.

7.13.4 Как Set понимает, что элемент уже есть?

Это зависит от реализации. В `HashSet` проверка идёт через хеш и равенство, потому что внутри используется `HashMap`. В `TreeSet` “одинаковость” определяется сравнением через `compareTo` или `Comparator`. Поэтому общий ответ такой: Set понимает дубликат через свой механизм сравнения, а не “на глаз”.

7.13.5 Когда Set подходит лучше List?

Когда по смыслу задачи повторы не нужны: уникальные логины, уникальные `id`, набор ролей, посещённые страницы, уникальные теги, множество уже обработанных значений. Если дубликаты в предметной области не имеют смысла, Set обычно честнее отражает модель данных, чем List.

7.14 HashSet

7.14.1 Что такое HashSet и на какой структуре он основан?

HashSet — это реализация Set, основанная на HashMap. В документации это сказано прямо, а в исходниках видно ещё нагляднее: внутри у HashSet есть поле map, и элементы множества хранятся как ключи этой карты, а значением служит специальный фиктивный объект PRESENT.

7.14.2 Почему HashSet обычно работает быстро?

Потому что вся его логика опирается на хеш-таблицу. Поиск, добавление и проверка наличия сводятся к операциям HashMap, а та при нормальном распределении хешей обычно работает очень быстро. Поэтому HashSet и воспринимается как “быстрое множество”.

7.14.3 Как HashSet определяет, есть ли элемент в наборе?

В исходниках это буквально делегировано в карту: contains(o) у HashSet вызывает map.containsKey(o). То есть элемент множества хранится как ключ в HashMap, и проверка наличия элемента — это проверка наличия ключа в карте.

Упрощённо по сути реализации:

```
boolean contains(Object o) {  
    return map.containsKey(o);  
}
```

7.14.4 Почему для HashSet так важны equals() и hashCode()?

Потому что HashSet не хранит элементы сам по себе — он хранит их через HashMap. Значит, всё, что критично для HashMap, автоматически критично и для HashSet: сначала используется хеш, потом равенство. Если эти методы реализованы плохо или несогласованно, множество начинает ломаться.

7.14.5 Что происходит при добавлении элемента в HashSet?

В реальной реализации add(e) почти буквальным образом превращается в map.put(e, PRESENT) == null. То есть множество пытается положить элемент как ключ в карту. Если ключа раньше не было, put возвращает null, и HashSet говорит, что элемент успешно добавлен. Если ключ уже был, множество остаётся без изменений.

Упрощённо:

```
boolean add(E e) {  
    return map.put(e, PRESENT) == null;  
}
```

7.14.6 Что такое коллизия в хеш-структурах?

Это ситуация, когда разные объекты дают один и тот же бакет. Для `HashSet` это не что-то отдельное от `HashMap`, потому что он полностью живёт поверх неё. Значит, коллизии здесь — это те же самые коллизии в хеш-таблице карты: несколько разных элементов оказываются в одном бакете, и тогда структура разбирает их дальше через цепочку или дерево.

7.14.7 Почему неправильный `equals()`/`hashCode()` ломает работу `HashSet`?

Потому что `HashSet` опирается на правило “одинаковые элементы должны попадать в одинаковую логическую ячейку и распознаваться как одинаковые”. Если равные по смыслу объекты дают разные хеши или `equals()` ведёт себя нестабильно, множество уже не может корректно понять, это дубликат или новый элемент. Итог — ложные дубликаты, невозможность найти элемент, странное удаление.

7.14.8 Можно ли менять объект после добавления в `HashSet`?

Если меняются поля, которые участвуют в `equals()` и `hashCode()`, это опасно. Формально объект уже лежит в множестве как ключ `HashMap`, и если его хеш или равенство меняются после вставки, структура может больше не находить его там, где он лежит. Поэтому изменяемые элементы в `HashSet` допустимы только тогда, когда изменения не влияют на сравнение и хеш. Это вывод напрямую из того, что `HashSet` хранит элемент как ключ в `HashMap`.

7.14.9 Чем в общих чертах `HashSet` отличается от `LinkedHashSet` и `TreeSet`?

`HashSet` — это быстрое множество без гарантии порядка обхода. `LinkedHashSet` — это `HashSet` с дополнительным двусвязным списком, поэтому у него есть предсказуемый порядок, обычно порядок вставки. `TreeSet` — это уже дерево, а не хеш-структура, поэтому элементы там отсортированы и основные операции стоят логарифмически.

7.15 Queue, Deque и PriorityQueue

7.15.1 Что такое очередь как структура данных?

Очередь — это структура, в которой элементы обычно добавляются для последующей обработки. Интерфейс `Queue` поверх обычных операций коллекции даёт отдельные операции вставки, извлечения и просмотра головы. Он прямо описывает себя как коллекцию для хранения элементов перед обработкой.

7.15.2 В чем идея FIFO?

FIFO означает **first in, first out** — “первым пришёл, первым вышел”. Для очереди это означает: новые элементы кладутся в хвост, а забираются из головы. В `Queue` прямо сказано, что очереди обычно, хотя и не всегда, работают по FIFO; исключения — например, очереди с приоритетом.

7.15.3 Что такое Deque?

`Deque` — это двусторонняя очередь. В ней можно вставлять, удалять и смотреть элементы **с обоих концов**. Само название и расшифровывается как *double ended queue*.

7.15.4 Чем очередь, дек и стек отличаются по поведению?

Обычная очередь обычно работает как FIFO: добавление в конец, удаление с начала. `Deque` умеет работать с обоих концов и может вести себя и как очередь, и как стек. Если дек используют как стек, получается LIFO: последний добавленный элемент снимается первым. Документация `Deque` прямо показывает соответствие методов очереди и стека его операциям `addFirst`, `addLast`, `removeFirst`, `pollFirst` и так далее.

7.15.5 Что такое очередь с приоритетом?

Очередь с приоритетом — это очередь, где важен не порядок поступления, а правило приоритета. В Java это `PriorityQueue`: она основана на **heap**-структуре и упорядочивает элементы либо по естественному порядку, либо по `Comparator`, переданному при создании. То есть первым извлекается не обязательно самый старый элемент, а тот, у кого выше приоритет по сравнению.

7.15.6 В каких задачах Queue, Deque и PriorityQueue особенно полезны?

Queue удобна для обычной последовательной обработки задач, событий, сообщений, запросов. Deque хороша там, где нужно эффективно работать с обоих концов или моделировать и очередь, и стек одной структурой.

PriorityQueue полезна, когда нужно всегда быстро доставать “самый важный” элемент: ближайшую задачу, минимальный/максимальный элемент, следующую по приоритету операцию. Эти роли хорошо совпадают с тем, как интерфейсы и классы определены и как они реализованы в Java.

ArrayDeque внутри хранит элементы в массиве и двигает указатели головы и хвоста. В реальной реализации addFirst уменьшает head по кругу и кладёт элемент туда, а addLast кладёт элемент в tail и сдвигает tail дальше. Это очень быстрая модель для работы по краям.

Упрощённо по сути реализации:

```
void addFirst(E e) {
    head = dec(head, elements.length);
    elements[head] = e;
    if (head == tail) grow();
}
```

```
void addLast(E e) {
    elements[tail] = e;
    tail = inc(tail, elements.length);
    if (head == tail) grow();
}
```

PriorityQueue внутри устроена иначе: это массив-heap. При offer(e) элемент добавляется в конец и поднимается вверх через siftUp, чтобы восстановить порядок кучи. При poll() берётся корень, последний элемент ставится наверх, а потом опускается вниз через siftDown. Именно поэтому очередь умеет быстро возвращать элемент с наивысшим приоритетом по своему правилу сравнения.

Упрощённо по сути реализации:

```
boolean offer(E e) {  
    if (size == queue.length) grow();  
    queue[size] = e;  
    siftUp(size, e);  
    size++;  
    return true;  
}
```

```
E peek() {  
    return queue[0];  
}
```

```
E poll() {  
    E result = queue[0];  
    E last = queue[--size];  
    queue[size] = null;  
  
    if (size > 0) {  
        siftDown(0, last);  
    }  
  
    return result;  
}
```

7.16 Итерация по коллекциям

7.16.1 Как пройти по коллекции в Java?

По коллекции в Java обычно проходят тремя базовыми способами: через `for - each`, через `Iterator` и через обычный индексный цикл, если структура вообще поддерживает нормальный доступ по индексу. Для `List` это особенно заметно: у списка есть и `iterator()`, и индексные методы, но для некоторых реализаций, например связанных списков, обход через индексы может быть заметно хуже обычной итерации.

Пример с `for - each`:

```
List<String> names = List.of("Анна", "Борис", "Вика");
```

```
for (String name : names) {  
    System.out.println(name);  
}
```

Пример с `Iterator`:

```
List<String> names = new ArrayList<>();  
names.add("Анна");  
names.add("Борис");  
names.add("Вика");
```

```
Iterator<String> it = names.iterator();  
while (it.hasNext()) {  
    String name = it.next();  
    System.out.println(name);  
}
```

7.16.2 Что такое Iterator?

Iterator — это объект-обходчик, который умеет последовательно проходить элементы коллекции. Идея в том, что сама коллекция даёт итератор через `iterator()`, а дальше мы двигаемся по элементам через `hasNext()` и `next()`. Для списков в Java базовая модель итератора описана даже в `AbstractList`: итератор опирается на `size()`, `get(int)` и `remove(int)`.

Упрощённо по сути это выглядит так:

```
Iterator<E> iterator() {  
    return new Itr();  
}
```

И дальше сам обходчик делает примерно следующее:

```
while (iterator.hasNext()) {  
    E value = iterator.next();  
    // работаем с value  
}
```

7.16.3 Зачем нужен Iterator, если есть foreach?

`foreach` удобен для простого чтения элементов, но `Iterator` даёт больше контроля: можно явно управлять обходом, безопасно удалять текущий элемент и работать с коллекцией через единый механизм, не привязываясь к конкретной реализации. По сути `for-each` — это просто более удобная запись поверх итератора. А для списков ещё есть `ListIterator`, который умеет идти в обе стороны, менять элементы и вставлять новые во время обхода.

7.16.4 Как безопасно удалять элементы во время обхода коллекции?

Безопасный способ — удалять через сам итератор, то есть через `iterator.remove()`. Тогда итератор знает, что структура изменилась именно через него, и корректно обновляет своё внутреннее состояние. В реальной реализации `LinkedList.ListItr.remove()` сначала вызывает `unlink(lastReturned)`, потом синхронизирует свои указатели и увеличивает `expectedModCount`.

Пример:

```
List<String> names = new ArrayList<>();
names.add("Анна");
names.add("Борис");
names.add("Вика");
```

```
Iterator<String> it = names.iterator();
while (it.hasNext()) {
    String name = it.next();
    if (name.startsWith("Б")) {
        it.remove();
    }
}
```

```
System.out.println(names); // [Анна, Вика]
```

Упрощённо по сути реализации идея такая:

```
public void remove() {
    checkForComodification();
    unlink(lastReturned); // или remove(lastIndex) у другой реализации
    lastReturned = null;
    expectedModCount++;
}
```

7.16.5 Почему нельзя просто удалять элементы из коллекции в цикле как попало?

Потому что итератор и сама коллекция должны быть согласованы по внутреннему состоянию. Если ты начал обход, а потом меняешь коллекцию напрямую не через итератор, итератор может увидеть, что структура уже не та, на которую он рассчитывал. В лучшем случае будет `ConcurrentModificationException`, в худшем — логика обхода начнёт вести себя странно. Java прямо описывает это как fail-fast поведение итераторов.

Плохой пример:

```
List<String> names = new ArrayList<>();
names.add("Анна");
names.add("Борис");
names.add("Вика");

for (String name : names) {
    if (name.startsWith("Б")) {
        names.remove(name); // так делать нельзя
    }
}
```

7.16.6 Что такое fail-fast поведение итераторов?

Это поведение, при котором итератор старается быстро заметить, что коллекцию изменили “мимо него”, и выбросить `ConcurrentModificationException`. У `ArrayList` это прямо оговорено: итераторы fail-fast, если список структурно изменён после создания итератора не через его собственные `remove()` или `add()`. Но важная деталь: это механизм для обнаружения багов, а не железная гарантия корректности многопоточной логики. В документации прямо сказано, что fail-fast работает по принципу best effort.

Упрощённо суть проверки такая:

```
final void checkForComodification() {
    if (modCount != expectedModCount) {
        throw new ConcurrentModificationException();
    }
}
```

7.16.7 Что такое `ListIterator` и чем он отличается от обычного `Iterator`?

`ListIterator` — это специальный итератор именно для списков. В отличие от обычного `Iterator`, он умеет идти не только вперёд, но и назад, а ещё поддерживает `set()` и `add()` во время обхода. В интерфейсе `List` это отдельно подчёркивается, а в реальной реализации `LinkedList.ListItr` видно, что у него есть `previous()`, `set(E e)` и `add(E e)`.

Пример:

```
List<String> names = new LinkedList<>();
names.add("Анна");
names.add("Борис");

ListIterator<String> it = names.listIterator();

while (it.hasNext()) {
    String value = it.next();

    if (value.equals("Анна")) {
        it.set("Анна П.");
        it.add("Вика");
    }
}

System.out.println(names); // [Анна П., Вика, Борис]
```


7.17 Arrays, Collections и неизменяемые коллекции

7.17.1 Что делает класс Arrays?

`Arrays` — это утилитный класс для работы именно с массивами. В нём собраны `static`-методы для сортировки, поиска, сравнения, заполнения, преобразования массива в список и других типовых операций. То есть `Arrays` не хранит данные сам по себе — он просто даёт готовые инструменты для массивов.

Примеры:

```
int[] numbers = {4, 2, 7, 1};
```

```
Arrays.sort(numbers);
```

```
System.out.println(Arrays.toString(numbers)); // [1, 2, 4, 7]
```

```
int index = Arrays.binarySearch(numbers, 4);
```

```
System.out.println(index); // 2
```

7.17.2 Что делает класс Collections?

`Collections` — это утилитный класс уже для работы с коллекциями. В нём лежат алгоритмы вроде сортировки, бинарного поиска, перемешивания, заполнения, а также разные обёртки: синхронизированные, непроверяемые по модификации, `read-only` и так далее. В описании класса прямо сказано, что он состоит только из `static`-методов, которые работают с коллекциями или возвращают коллекции.

Примеры:

```
List<String> names = new ArrayList<>();
```

```
names.add("Вика");
```

```
names.add("Анна");
```

```
names.add("Борис");
```

```
Collections.sort(names);
```

```
System.out.println(names); // [Анна, Борис, Вика]
```

```
Collections.shuffle(names);
```

```
System.out.println(names); // порядок случайный
```

Идея некоторых методов видна прямо в коде. Например,

```
Collections.sort(list, c)
```

 сейчас просто делегирует в `list.sort(c)`. А

`binarySearch` выбирает один из двух вариантов: для `RandomAccess`-списков идёт по индексам, а для последовательных списков — через `ListIterator`.

Упрощённо:

```
public static <T> void sort(List<T> list, Comparator<? super T> c) {
    list.sort(c);
}

public static <T> int binarySearch(List<? extends Comparable<? super T>> list, T
key) {
    if (list instanceof RandomAccess || list.size() < THRESHOLD) {
        return indexedBinarySearch(list, key);
    } else {
        return iteratorBinarySearch(list, key);
    }
}
```

7.17.3 Что такое неизменяемая коллекция?

Неизменяемая коллекция — это коллекция, в которую нельзя добавить, удалить или заменить элемент через её API. После создания она держит тот же набор ссылок на элементы. В Java для таких коллекций есть удобные фабрики `List.of`, `Set.of`, `Map.of`, а в документации они описаны как `unmodifiable`.

Пример:

```
List<String> names = List.of("Анна", "Борис", "Вика");

// names.add("Глеб"); // UnsupportedOperationException
```

7.17.4 Чем неизменяемая коллекция отличается от изменяемой?

Изменяемая коллекция разрешает операции вроде `add`, `remove`, `set`, `clear`.

Неизменяемая — нет. Внутри реализаций `immutable`-коллекций это видно совсем прямо: методы изменения просто бросают

`UnsupportedOperationException`. В `ImmutableCollections` буквально есть методы вида `add(...)` { `throw uoe();` }, `remove(...)` { `throw uoe();` }, `set(...)` { `throw uoe();` }.

Упрощённо по сути реализации:

```
@Override
public boolean add(E e) {
    throw new UnsupportedOperationException();
}
```

```
@Override
public E remove(int index) {
    throw new UnsupportedOperationException();
}
```

7.17.5 Зачем нужны `unmodifiable`-обёртки?

Они нужны, чтобы дать внешнему коду доступ только на чтение, не отдавая возможность менять внутреннюю коллекцию напрямую. Это особенно полезно, когда класс хранит коллекцию внутри себя и хочет показать её наружу безопасно.

При этом важно понимать: `Collections.unmodifiableList(list)` — это именно **вид** поверх существующей коллекции, а не обязательно отдельная независимая копия. Изменения в `backing`-коллекции остаются видны через такую обёртку.

Пример:

```
List<String> original = new ArrayList<>();
original.add("A");
original.add("B");

List<String> readOnly = Collections.unmodifiableList(original);
// readOnly.add("C"); // UnsupportedOperationException
original.add("C");
System.out.println(readOnly); // [A, B, C]
```

7.17.6 Почему неизменяемые коллекции полезны для надежности кода?

Потому что они уменьшают число неожиданных изменений состояния. Когда ты передаёшь коллекцию дальше по коду и точно знаешь, что её нельзя мутировать, поведение становится предсказуемее. Это особенно важно для API, DTO, конфигурации, кэшей справочников и в целом там, где коллекция должна быть “данными, а не рабочей площадкой для изменений”. При этом важно различать `unmodifiable` и `truly immutable`: если сами элементы изменяемые, коллекция может оставаться логически подвижной, даже если её методы мутации запрещены.

7.18 Сложность операций и выбор коллекции

7.18.1 Что такое временная сложность операций?

Временная сложность — это способ описать, как растёт стоимость операции при росте количества данных. Это не “точное число миллисекунд”, а модель роста. Например, одни операции растут почти незаметно, другие — пропорционально размеру структуры, третьи — логарифмически. В выборе коллекции эта идея важна, потому что даже одинаковый по смыслу код на разных структурах данных начинает вести себя по-разному на больших объёмах.

7.18.2 Почему при выборе коллекции важно понимать ее внутреннюю реализацию?

Потому что внешне две структуры могут выглядеть похоже, а внутри быть устроены совершенно по-разному. Например, `ArrayList` и `LinkedList` оба реализуют `List`, но один основан на массиве, а второй — на двусвязном списке. Из-за этого операции по индексам, вставки, удаления и обход работают с разной ценой. То же самое для `HashMap` и `TreeMap`: обе — карты, но внутри это хеш-таблица и дерево.

7.18.3 Какие операции важнее всего учитывать при выборе структуры данных?

Обычно нужно смотреть прежде всего на то, что в задаче происходит чаще: чтение по индексу, поиск по ключу, проверка наличия, добавление в конец, вставка в середину, удаление, обход, сохранение порядка, сортировка, уникальность, работа с головой очереди. Именно частые операции важнее всего, потому что они и определяют реальную стоимость структуры данных в проекте.

7.18.4 Почему нет одной лучшей коллекции на все случаи жизни?

Потому что сильная сторона одной структуры почти всегда оплачивается слабой стороной в другой операции. `ArrayList` удобен для индексов и обхода, `HashSet` — для уникальности и быстрой проверки наличия, `HashMap` — для доступа по ключу, `TreeMap` и `TreeSet` — для отсортированности, `PriorityQueue` — для быстрого извлечения элемента по приоритету. Одна “идеальная” структура на всё сразу либо была бы слишком сложной, либо проигрывала бы специализированным решениям.

7.18.5 Как выбирать между List, Set, Queue и Map?

Логика выбора простая:

если нужна **последовательность** и возможны повторы — List;

если важна **уникальность** — Set;

если нужна модель **очереди обработки** — Queue или Deque;

если данные естественно живут как **ключ** → **значение** — Map.

Сначала выбирают именно абстракцию задачи, а уже потом конкретную реализацию.

7.18.6 Как выбирать между ArrayList, HashSet, HashMap и другими популярными реализациями?

ArrayList — когда нужен список, порядок и удобный доступ по индексу.

LinkedList — когда структура реально похожа на дек или очередь с работой по краям.

HashSet — когда нужны уникальные элементы и быстрая проверка наличия.

LinkedHashSet — когда ещё важен стабильный порядок обхода.

TreeSet — когда нужен отсортированный набор.

HashMap — когда нужна быстрая карта по ключу.

LinkedHashMap — когда важен порядок обхода записей.

TreeMap — когда ключи должны быть отсортированы.

PriorityQueue — когда нужен быстрый доступ не к “первому вставленному”, а к “наиболее приоритетному”. Это соответствует и их устройству, и

характеристикам, которые оговариваются в документации: у HashMap и HashSet базовые операции в среднем константные при хорошем хеше, у TreeMap и

TreeSet операции логарифмические, у PriorityQueue внутри heap-структура.

8. Исключения в Java

8.1 Что такое исключение в Java?

Исключение — это объект, который описывает ошибочную или особую ситуацию во время выполнения программы. В Java бросать и ловить можно только объекты типа `Throwable` и его наследников. При этом `Exception` и её наследники — это ситуации, которые разумное приложение может захотеть обработать.

Пример:

```
int a = 10;  
int b = 0;
```

```
int result = a / b; // ArithmeticException
```

8.2 Зачем в программе вообще нужен механизм исключений?

Он нужен, чтобы не смешивать основной код с постоянными ручными проверками ошибок и чтобы можно было передать информацию о проблеме вверх по стеку вызовов туда, где её действительно умеют обработать. Исключения отделяют нормальный сценарий работы от сценария ошибки и позволяют не потерять контекст проблемы.

8.3 Чем `Exception` отличается от `Error`?

`Exception` описывает ситуации, которые приложение может захотеть обработать. `Error` описывает серьёзные проблемы, которые разумная программа обычно не должна пытаться ловить и лечить обычной бизнес-логикой. В Java это разделение зафиксировано прямо в API: `Exception` — это то, что приложение может захотеть поймать, а `Error` — это серьёзные проблемы.

8.4 Что такое `checked` и `unchecked` исключения?

Checked-исключения компилятор заставляет либо обработать, либо объявить в `throws`.

Unchecked-исключения — это `RuntimeException`, `Error` и их наследники; их не нужно объявлять в сигнатуре и компилятор не требует обязательно их ловить. В Java это прямо оговорено в обучающих материалах по исключениям.

8.5 Чем RuntimeException отличается от остальных исключений?

RuntimeException — это подтип Exception, но он относится к unchecked-исключениям. Его и его наследников не нужно объявлять в throws, если они могут вылететь из метода. Обычно это ошибки использования API, нарушения предусловий, проблемы логики и подобные вещи, которые часто проявляются уже во время выполнения.

8.6 Почему возникает NullPointerException?

Он возникает, когда код пытается использовать null там, где нужен объект. Например: вызвать метод у null, обратиться к полю null, взять длину у null как у массива, обращаться к элементу null-массива. В API NullPointerException эти случаи перечислены прямо.

Пример:

```
String text = null;  
System.out.println(text.length()); // NullPointerException
```


8.7 Что делают try, catch и finally?

try содержит код, где может произойти исключение. catch перехватывает исключение определённого типа и позволяет отреагировать на него. finally выполняется при выходе из try и нужен прежде всего для завершающих действий, например очистки ресурсов. Для finally отдельно подчёркивается, что он помогает не пропустить cleanup даже если случилась ошибка, return, break или continue.

Пример:

```
try {
    System.out.println("Начало");
    int x = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Поймали ошибку: " + e.getMessage());
} finally {
    System.out.println("finally выполнится всегда");
}
```

8.8 Чем throw отличается от throws?

throw — это команда **бросить конкретное исключение прямо сейчас**.

throws — это часть сигнатуры метода, которая говорит: “этот метод может выбросить такие-то checked-исключения”. То есть throw работает в теле метода, а throws — в объявлении метода.

Пример:

```
public void validateAge(int age) {
    if (age < 0) {
        throw new IllegalArgumentException("Возраст не может быть отрицательным");
    }
}

public String readFile() throws IOException {
    return Files.readString(Path.of("test.txt"));
}
```

8.9 Что значит, что исключение всплывает по стеку вызовов?

Это значит, что если текущий метод не поймал исключение, управление выходит из него и исключение идёт выше — в вызвавший метод. Если и там нет подходящего `catch`, оно идёт ещё выше. Так продолжается, пока исключение либо не будет обработано, либо не дойдёт до границы потока выполнения и не завершит его с ошибкой.

Пример:

```
public static void c() {  
    throw new RuntimeException("Ошибка в c()");  
}
```

```
public static void b() {  
    c();  
}
```

```
public static void a() {  
    b();  
}
```

Если вызвать `a()`, исключение сначала возникнет в `c()`, потом “поднимется” в `b()`, потом в `a()`.

8.10 Что такое stack trace и как его читать?

stack trace — это список кадров стека вызовов в момент ошибки. В Throwable есть методы `getStackTrace()` и `printStackTrace()`, которые как раз дают этот след. Читать его удобно так: сначала смотришь на тип исключения и сообщение, потом ищешь первую строку из **своего** кода, где реально произошёл сбой, а дальше идёшь вверх по цепочке вызовов и понимаешь, как программа до туда дошла.

Пример:

```
try {
    String s = null;
    s.length();
} catch (Exception e) {
    e.printStackTrace();
}
```

8.11 Когда исключение нужно обработать, а когда лучше пробросить выше?

Обрабатывать исключение стоит там, где ты действительно можешь **что-то осмысленно сделать**: восстановиться, выбрать запасной сценарий, вернуть корректный ответ пользователю, закрыть локальную задачу. Если текущий уровень не знает, как правильно реагировать, лучше не “глотать” проблему, а пробросить её выше. Для checked-исключений это прямо выражено через `throws`: если метод не ловит checked-исключение, он обязан объявить его в сигнатуре.

8.12 Зачем создавать собственные исключения?

Собственные исключения нужны, когда хочется выразить проблему на языке предметной области, а не бросать всё подряд как `RuntimeException("что-то пошло не так")`. Например, `InvalidOrderStateException`, `UserAlreadyExistsException`, `PaymentRejectedException` намного лучше показывают смысл ошибки и делают код понятнее. Это особенно полезно на границах слоёв и в бизнес-логике.

Пример:

```
public class UserAlreadyExistsException extends RuntimeException {
    public UserAlreadyExistsException(String email) {
        super("Пользователь уже существует: " + email);
    }
}
```

8.13 Почему важно не терять исходную причину ошибки при оборачивании исключений?

Потому что иначе теряется реальный источник проблемы. В `Throwable` есть понятие **cause** — исходной причины исключения. Если ты оборачиваешь низкоуровневую ошибку в более подходящее бизнес-исключение, важно передать внутрь старое исключение как `cause`, чтобы потом можно было восстановить полную картину. Иначе в логах останется только “верхняя упаковка”, а корень проблемы пропадёт.

Пример:

```
try {
    Files.readString(Path.of("config.txt"));
} catch (IOException e) {
    throw new IllegalStateException("Не удалось загрузить конфиг", e);
}
```

8.14 Какие плохие практики чаще всего встречаются при работе с исключениями?

Самые частые плохие практики такие:

ловить `Exception` или `Throwable` без необходимости;

глотать исключение и ничего не делать;

использовать исключения как обычную ветку логики;

ловить `NullPointerException` вместо нормальной проверки условий;

писать слишком общие и бессмысленные сообщения вроде "error";

Практически все эти проблемы сводятся к одной мысли: исключение надо либо действительно обработать, либо честно передать выше, сохранив контекст.

Плохой пример:

```
try {
    service.process();
} catch (Exception e) {
    // ничего не делаем
}
```

Лучше хотя бы так:

```
try {
    service.process();
} catch (IOException e) {
    throw new IllegalStateException("Не удалось обработать файл", e);
}
```

8.15 Что такое ресурс, что такое try-with-resources и как с этим работать?

Ресурс — это объект, который во время работы программы удерживает внешний системный ресурс и поэтому должен быть корректно закрыт после использования. Это могут быть файлы, потоки ввода-вывода, сокет, JDBC-объекты и другие подобные вещи. В Java ресурсом для try-with-resources считается объект, который реализует интерфейс `AutoCloseable`. Интерфейс `Closeable` — это его частный случай, который чаще встречается в I/O-коде.

`AutoCloseable` — это общий интерфейс для объектов, которые нужно закрывать. У него есть метод `close()`. Именно поэтому try-with-resources умеет автоматически закрывать такие объекты в конце блока try. `Closeable` расширяет `AutoCloseable` и используется в классах вроде `BufferedReader`, `FileInputStream`, `FileOutputStream`. Для него отдельно указано, что повторный вызов `close()` не должен иметь эффекта.

Смысл try-with-resources в том, что ресурсы объявляются прямо в заголовке try, а Java сама вызывает `close()` автоматически, даже если внутри блока произошло исключение. Если ресурсов несколько, они закрываются **в обратном порядке создания**. Это безопаснее и короче, чем ручное закрытие через `finally`.

Пример:

```
try (BufferedReader reader = Files.newBufferedReader(Path.of("data.txt"))) {  
    String line = reader.readLine();  
    System.out.println(line);  
} catch (IOException e) {  
    e.printStackTrace();  
}
```

Здесь `BufferedReader` подходит для try-with-resources, потому что реализует `Closeable`, а значит и `AutoCloseable`. После выхода из блока try Java сама вызовет `reader.close()`.

try-with-resources удобнее обычного `finally` ещё и потому, что правильно работает с двумя ошибками сразу. Если одна ошибка произошла в основном коде, а вторая — при закрытии ресурса, основной ошибкой считается та, что возникла в блоке try, а ошибка закрытия не теряется, а попадает в **suppressed exceptions**. Это можно получить через `getSuppressed()`. В старом варианте с `finally` была обратная проблема: исключение из `close()` могло затереть исходную причину.